


AINC
Augmented Intelligence & Cognition Lab.

# 딥러닝 응용

한신대학교
AI·SW대학
김선만

# 실습 : 모델 구조 설계

최대한 성능을 향상시켜보자.

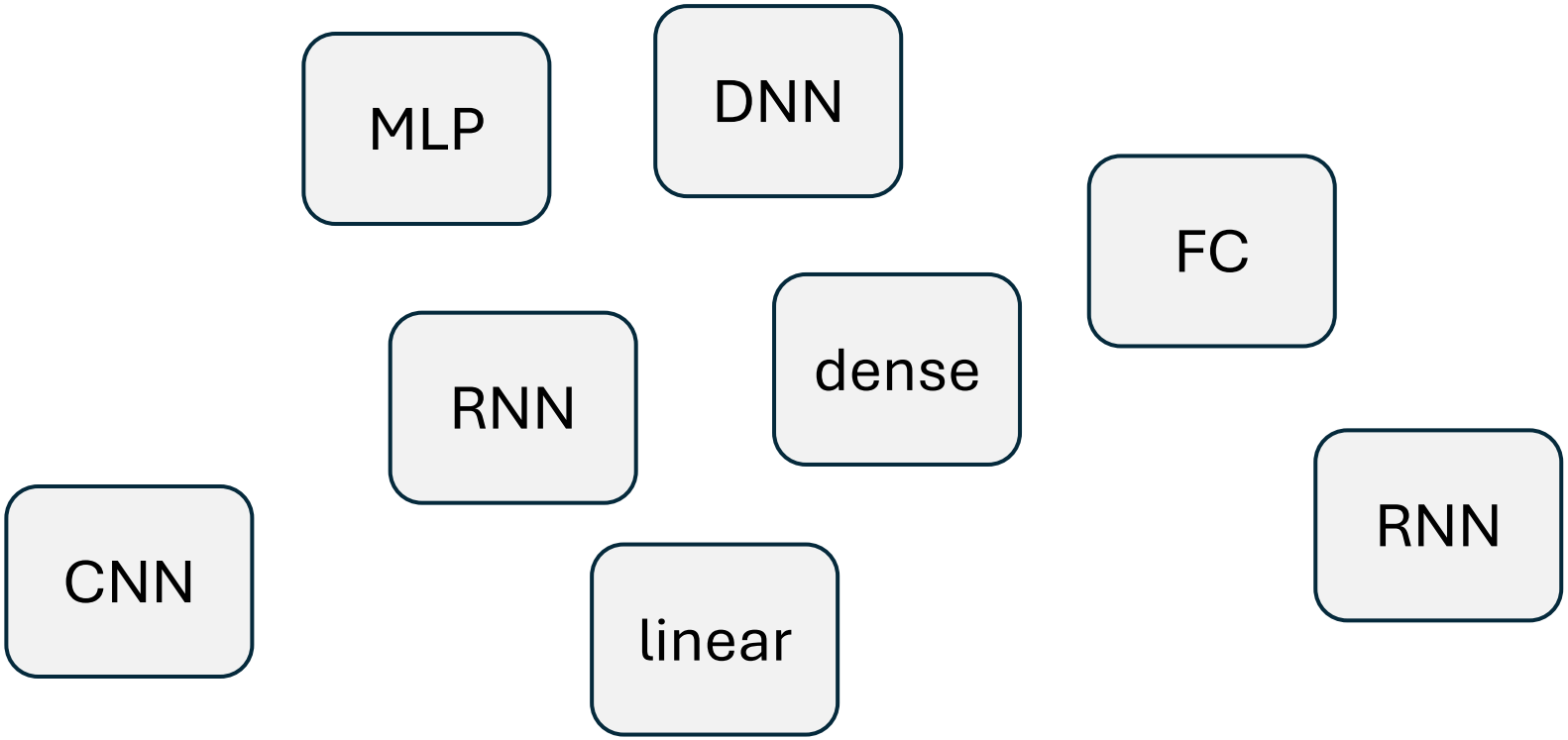
파라미터수 & 연산량은 어떻게 되지?

# CNN맛보기

model_v3.py

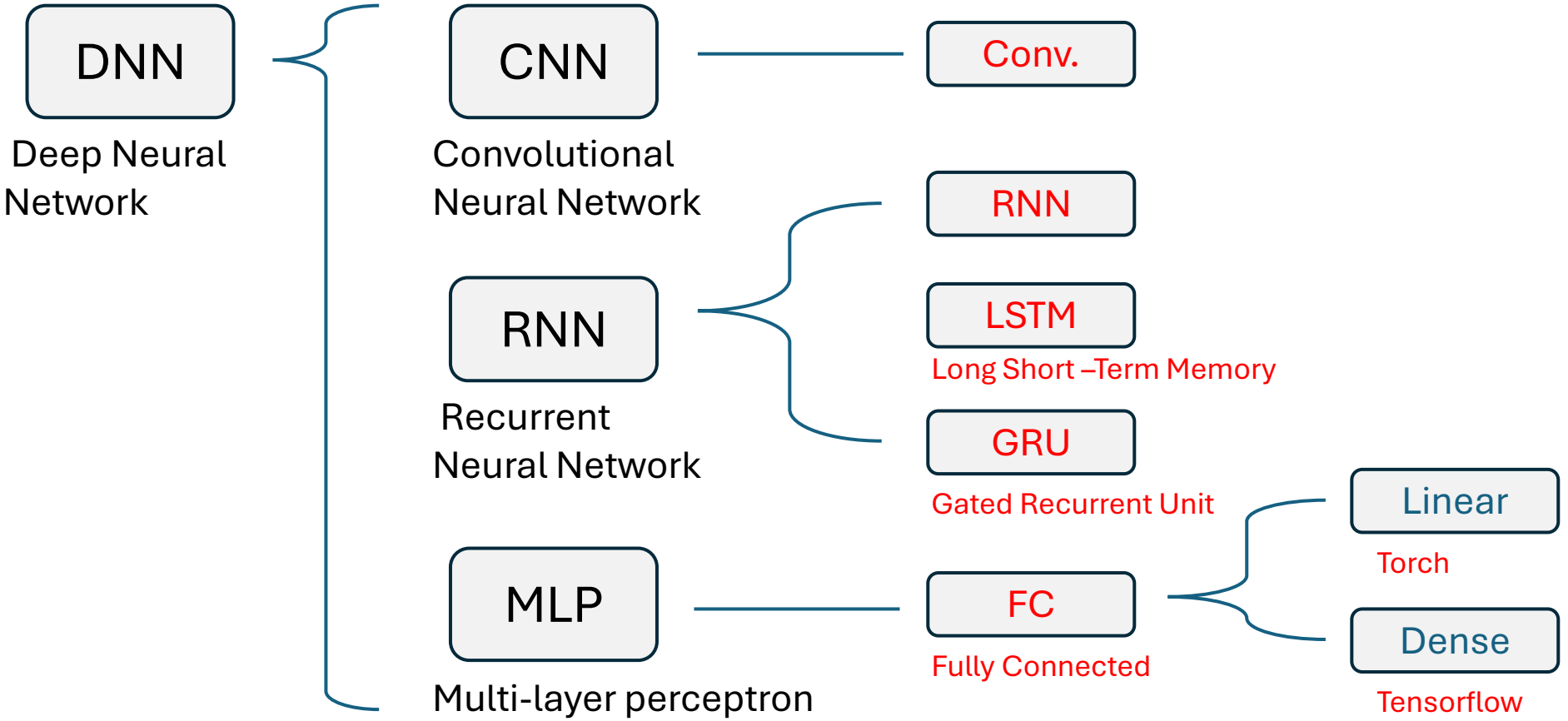
# 기본 심층신경망 구조 및 개념 정리

AINC
Augmented Intelligence & Cognition Lab.



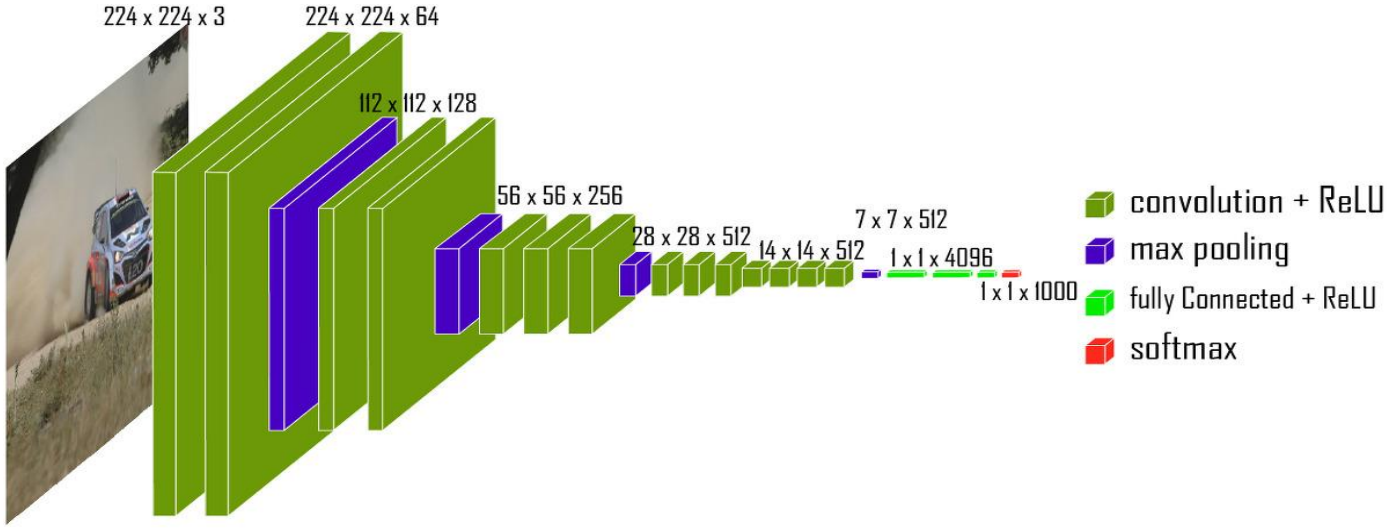
# 기본 심층신경망 구조 및 개념 정리

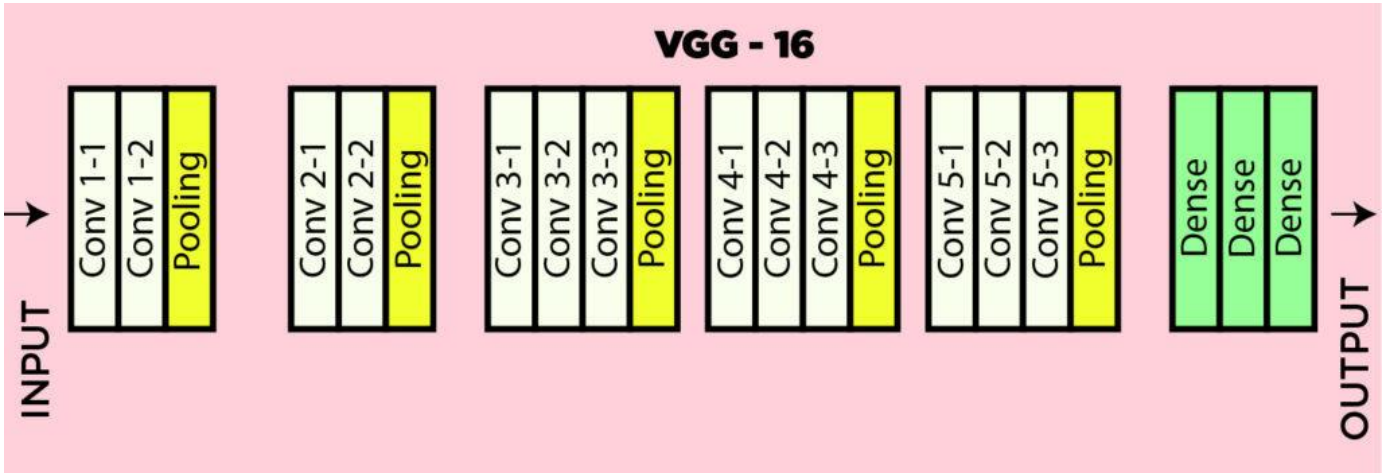
AINC
Augmented Intelligence & Cognition Lab.



# 기본 심층신경망 구조 및 개념 정리

AINC
Augmented Intelligence & Cognition Lab.




레이어 수는 몇 개?

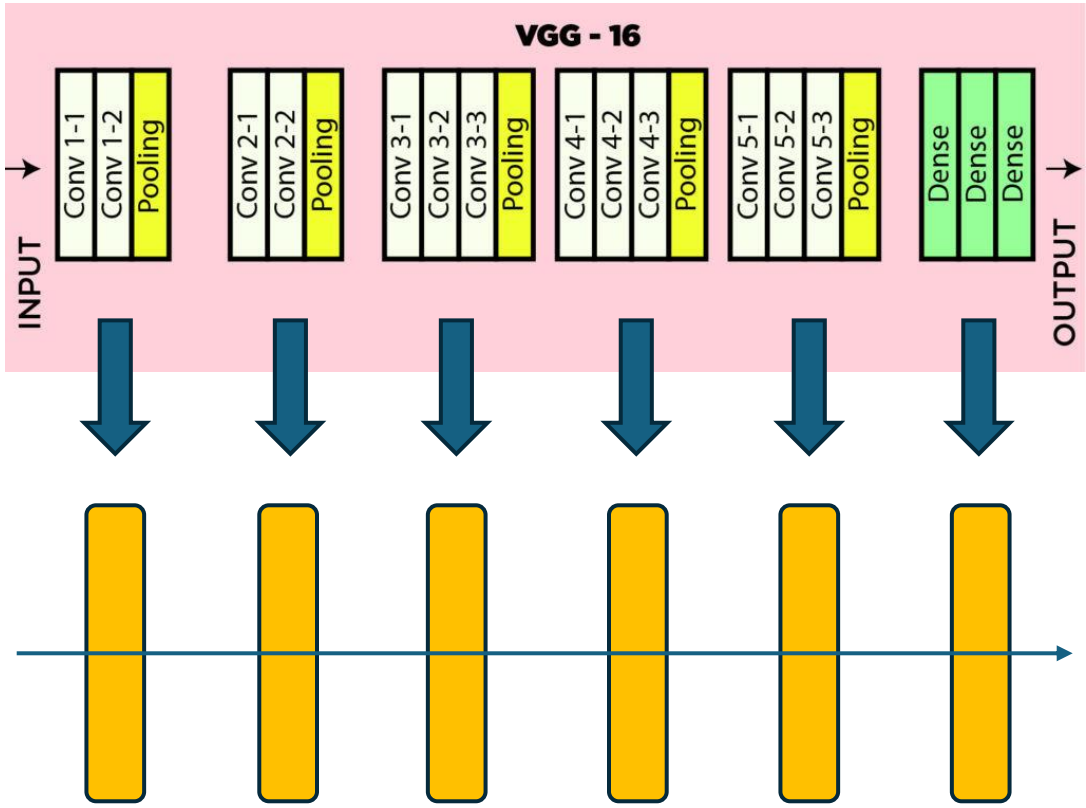
https://lcyking.tistory.com/entry/VGG-%EB%AA%A8%EB%8D%B8%EC%9D%98-%EA%B0%9C%EC%9A%94-%EB%B0%8F-%ED%8A%B9%EC%A7%95

AINC
Augmented Intelligence & Cognition Lab.

## Sequential Layer ?

복잡한 모델 내에서 반복되는 레이어 그룹을 하나의 sequential 블록으로 묶어 사용

데이터 흐름이 입력부터 출력까지 하나의 경로로 이루어진 모델을 만들 때 유용



# 기본 심층신경망 구조 및 개념 정리

AINC
Augmented Intelligence & Cognition Lab.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
#%%
class MLP(nn.Module):
    def __init__(self, in_dim: int):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, 256)
        self.fc2 = nn.Linear(256,     256)
        self.fc3 = nn.Linear(256,     10)

    def forward(self, x):
        x1 = self.fc1(x)
        x1 = F.relu(x1)
        x2 = self.fc2(x1)
        x2 = F.relu(x2)
        x3 = self.fc3(x2)
        return x3

#%%
if __name__ == "__main__":
    batch_size = 8
    in_dim = 784  # 예: 28x28 이미지를 펼친 크기
    x = torch.randn(batch_size, in_dim)
    model = MLP(in_dim)
    output = model(x)

    print(f"Input shape:  {x.shape}")
    print(f"Output shape: {output.shape}")
```

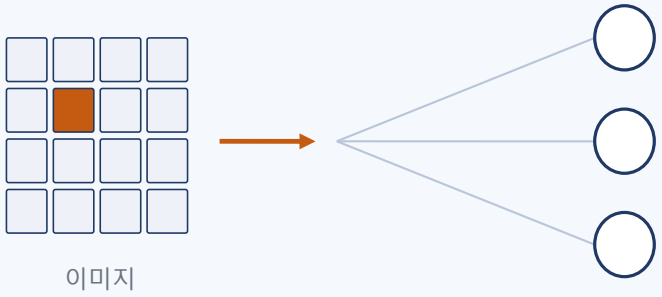
```python
import torch.nn as nn
import torch
#%%
class MLP(nn.Module):
    def __init__(self, in_dim):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(in_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        return self.layers(x)
#%%
if __name__ == "__main__":
    batch_size = 8
    in_dim = 784  # 예: 28x28 이미지를 펼친 크기
    x = torch.randn(batch_size, in_dim)
    model = MLP(in_dim)
    output = model(x)

    print(f"Input shape:  {x.shape}")
    print(f"Output shape: {output.shape}")
```

실습에 들어가기 전 근본 질문 하나 — 이미지를 그냥 전결합(FC)으로 학습하면 무슨 일이?

## ① 이미지를 펼쳐 FC에 연결



이미지

모든 픽셀 ↔ 모든 뉴런 (전결합)

## ② 파라미터 폭발

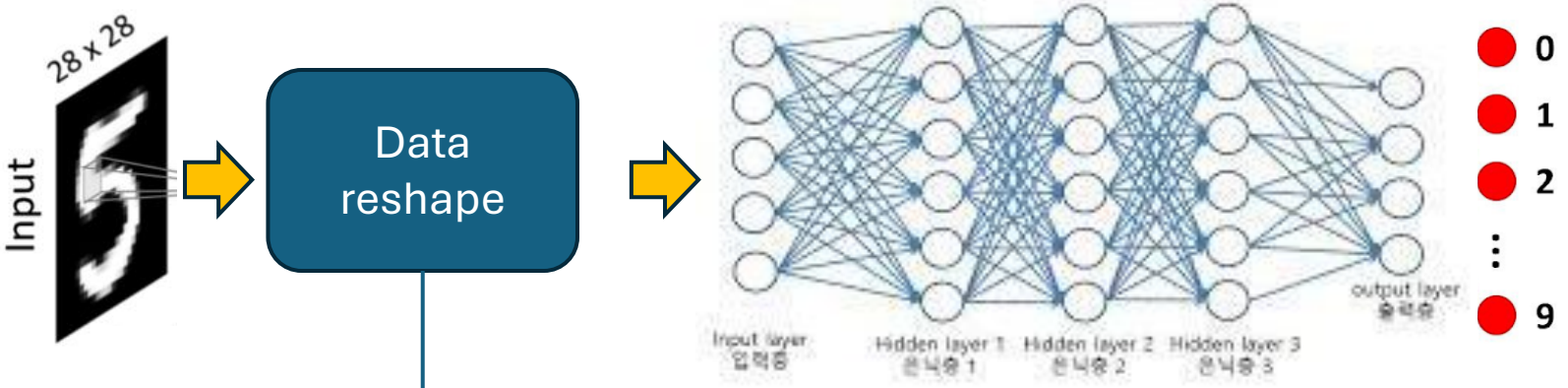
200 × 200 이미지 = **40,000** 픽셀

은닉 유닛 **40,000** 개

**40,000 × 40,000 ≈ 16억 (1.6B)**

단 한 층에서만! · 데이터·연산비용 폭증

**이웃 픽셀의 공간 구조는 살리고 파라미터는 줄이는 길 → 합성곱(Convolution)**



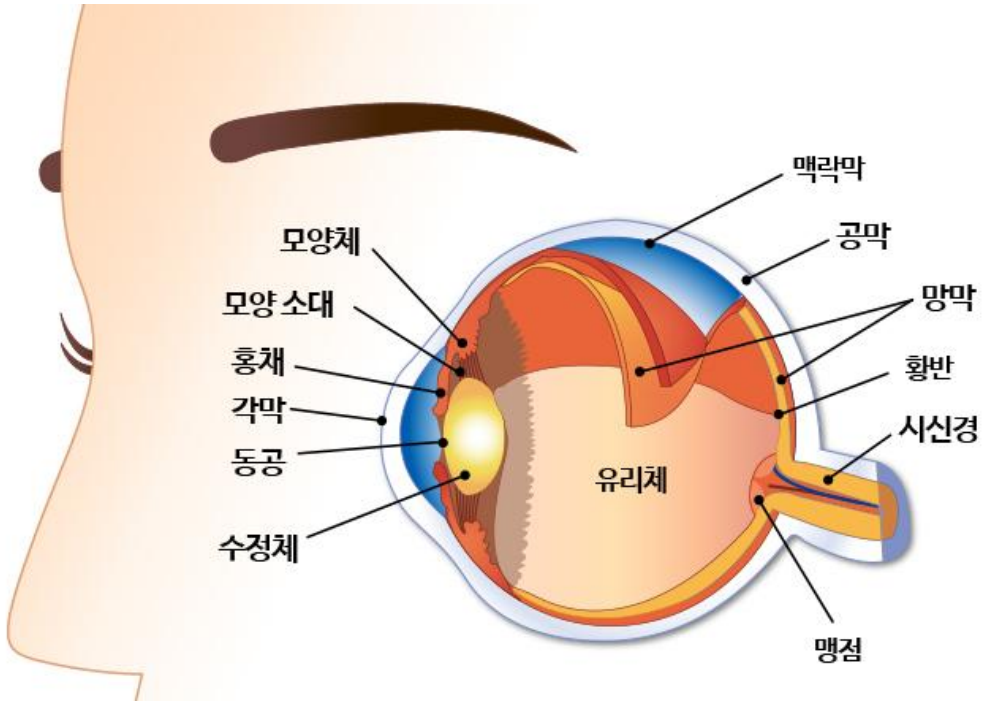

```
12
13      # 2) 다운사이징(7×7) 혹은 원본(28×28)
14      if use_small:
15          x_train_in = x_train[:, ::4, ::4].reshape(-1, 7*7)
16          x_test_in  = x_test[:,  ::4, ::4].reshape(-1, 7*7)
17      else:
18          x_train_in = x_train.reshape(-1, 28*28)
19          x_test_in  = x_test.reshape(-1, 28*28)
20
```

data_loader.py

사람은 어떻게 이미지 색을 인지하지?

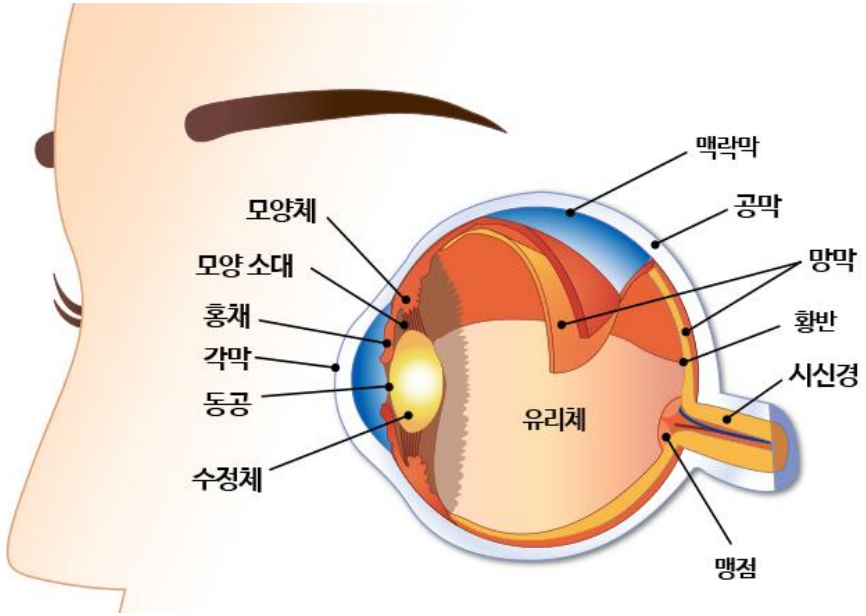
사람은 어떻게 특정 형상을 인지하지?

# ※ 시각 메커니즘




https://www.edunet.net/nedu/gmediaVideo/index-keris.do?cid=570175&autoPlay=false&nVolume=60&startTime=100&endTime=200

# ※ 이미지 데이터 & 행렬




사람의 색 구분 세포

-적색(Red) 감지

-녹색(Green) 감지

-청색(Blue) 감지

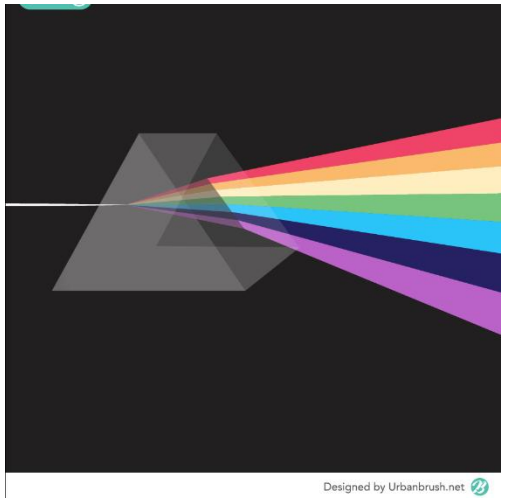
3개의 세포 각각이 어느 정도의 세기로 자극 되냐에 따라, 인간이 느끼는 색이 결정

https://m.blog.naver.com/apd142/221306122034

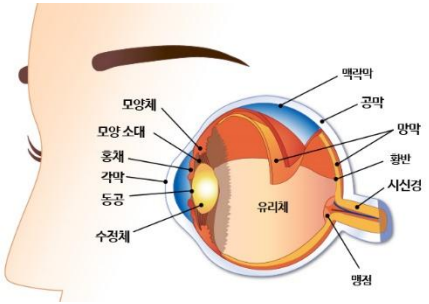
# ※ 빛의 삼원색



# ※ 빛의 삼원색

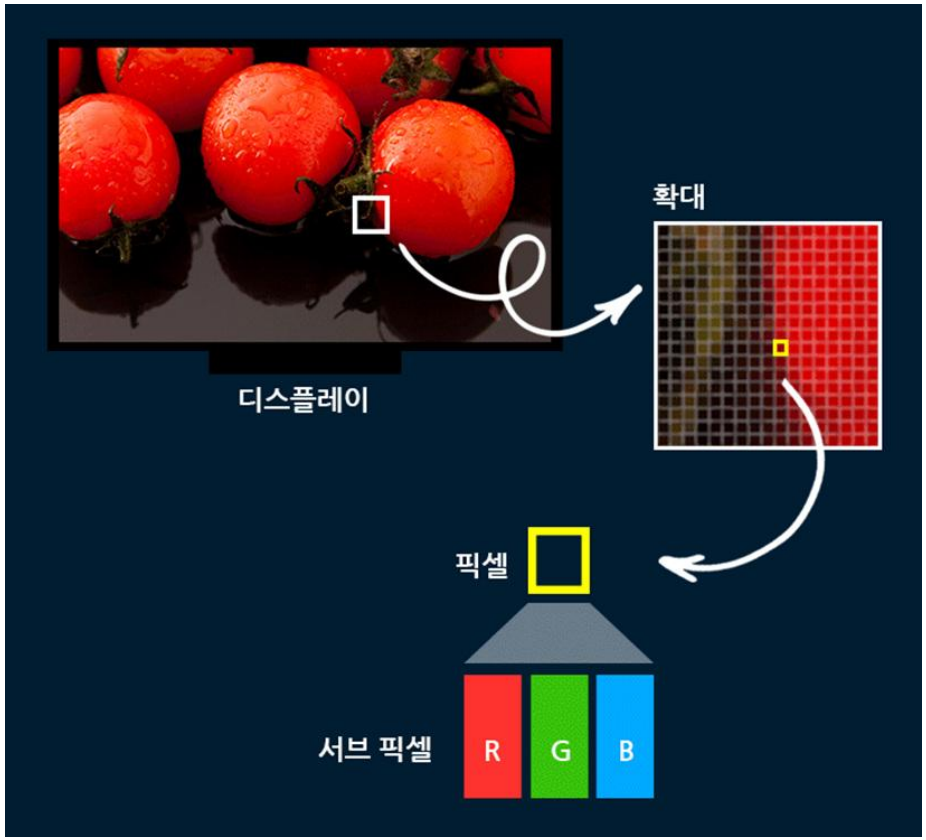





https://news.samsungdisplay.com/24640

# ※ 디스플레이




'갤럭시 S20 울트라'

3200 X 1440의 해상도

--> 4,608,000개의 픽셀

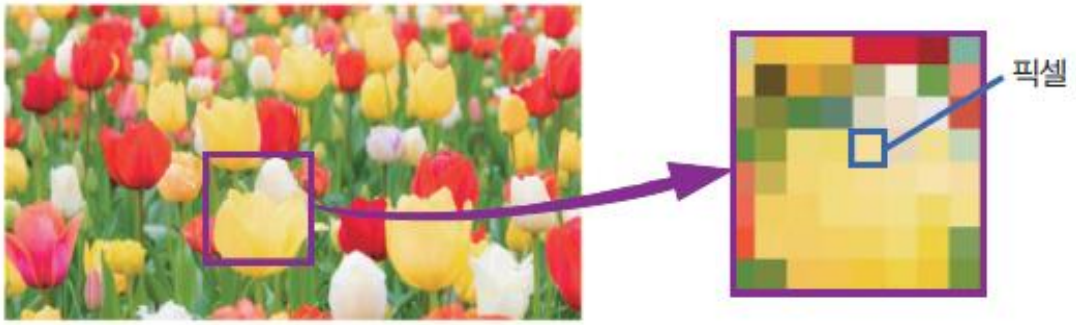
https://news.samsungdisplay.com/24640

# 흑백 이미지 데이터의 행렬 표현

- 사람의 뇌의 여러 가지 기능을 소프트웨어로 구현한 인공지능은 다양한 이미지 데이터를 행렬(Matrix)를 이용하여 디지털 정보(Digital Information)로 인식하여 표현한다.

## ① 픽셀의 정의

- 스마트폰에 내장된 카메라로 찍은 사진 이미지를 수평선과 수직선으로 분할하면 사진 이미지는 격자 모양의 작은 사각형으로 구성된다. 이 작은 사각형을 픽셀(Pixel)이라고 부른다.

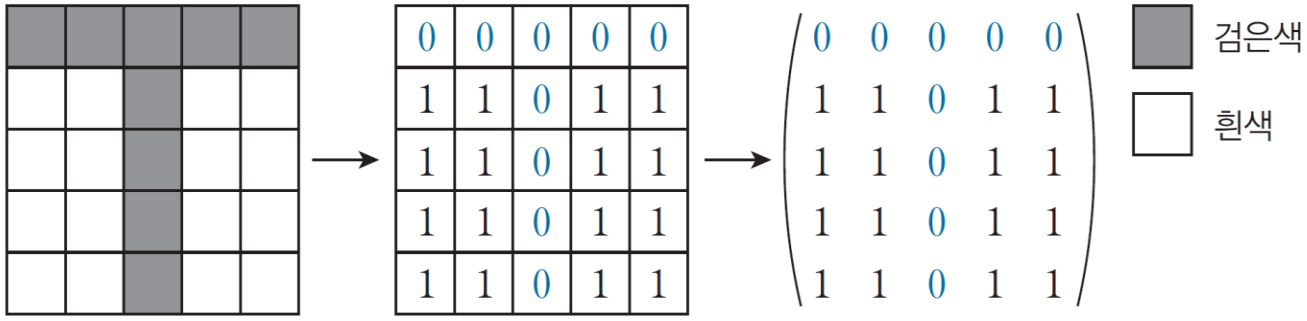



[그림 2.24] 픽셀의 개념

- 이미지에서 픽셀은 색의 정보를 가지고 있으며, 색의 정보를 수치화 할 수 있다.

  → 결과적으로 어떤 이미지는 픽셀들의 모임이며, 각각의 픽셀들은 수치화된 색의 정보를 가지고 있기 때문에 인공지능은 이를 인식하여 주어진 이미지가 무엇인지를 판단할 수 있다.

- 검은색은 0, 흰색은 1의 숫자를 부여

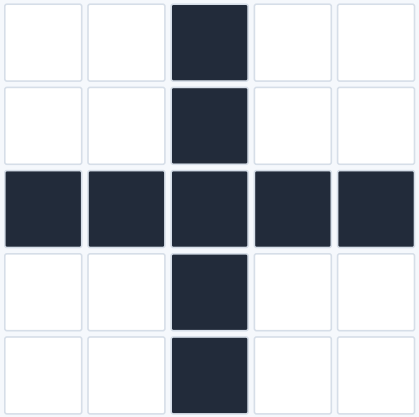



[그림 2.25] 흑백 이미지 데이터의 행렬 표현

# 이미지 = 행렬 (흑백)

이미지를 격자로 나눈 한 칸이 '픽셀' — 흑백 이미지는 0과 1의 행렬

## 이미지 (픽셀)



검은색 = 0, 흰색 = 1

## 행렬 표현

| | | | | |
|---|---|---|---|---|
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 |

0 이 모여 '十' 모양을 이룬다

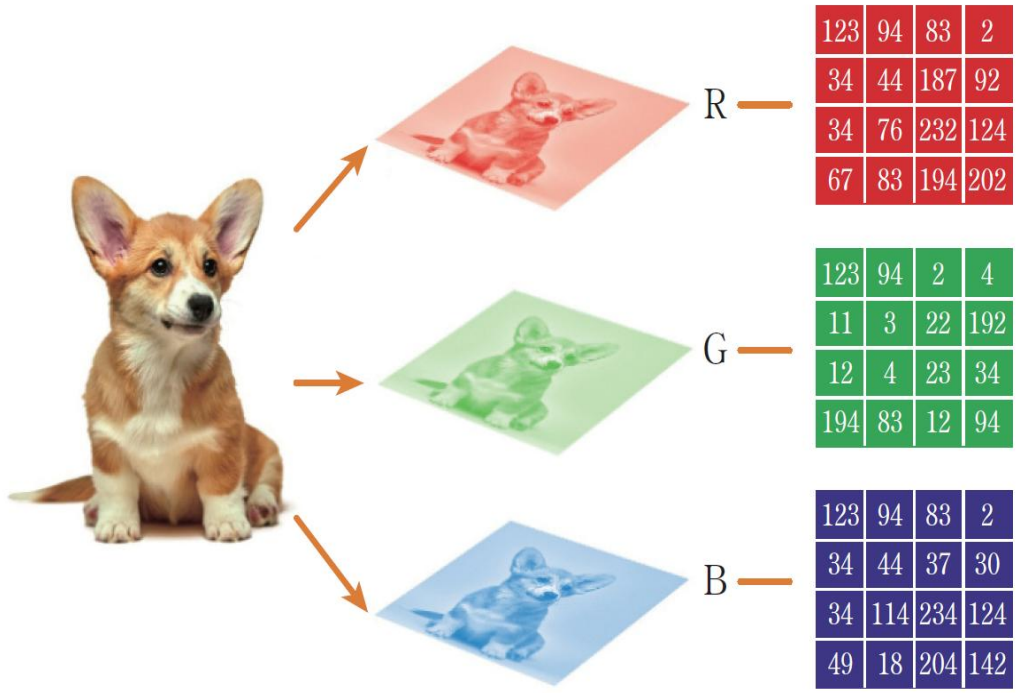
**픽셀값 = 밝기.** 신경망에 입력되는 것은 이 숫자 행렬입니다 (보통 0~255 또는 0~1로 정규화).

- 컬러 이미지의 3차원 색 정보 표현 : 각 픽셀의 색 정보가 $r_{ij}, g_{ij}, b_{ij}$로 표현되기 때문에 다음 3개의 행렬로 표현 가능

$$\boldsymbol{R} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & r_{14} \\ r_{21} & r_{22} & r_{23} & r_{24} \\ r_{31} & r_{32} & r_{33} & r_{34} \\ r_{41} & r_{42} & r_{43} & r_{44} \end{pmatrix} = (r_{ij})$$

$$\boldsymbol{G} = \begin{pmatrix} g_{11} & g_{12} & g_{13} & g_{14} \\ g_{21} & g_{22} & g_{23} & g_{24} \\ g_{31} & g_{32} & g_{33} & g_{34} \\ g_{41} & g_{42} & g_{43} & g_{44} \end{pmatrix} = (g_{ij})$$

$$\boldsymbol{B} = \begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{pmatrix} = (b_{ij})$$




[그림 2.29] 컬러 이미지의 3차원 색 정보 표현

# A bit of history:

**Hubel & Wiesel,**

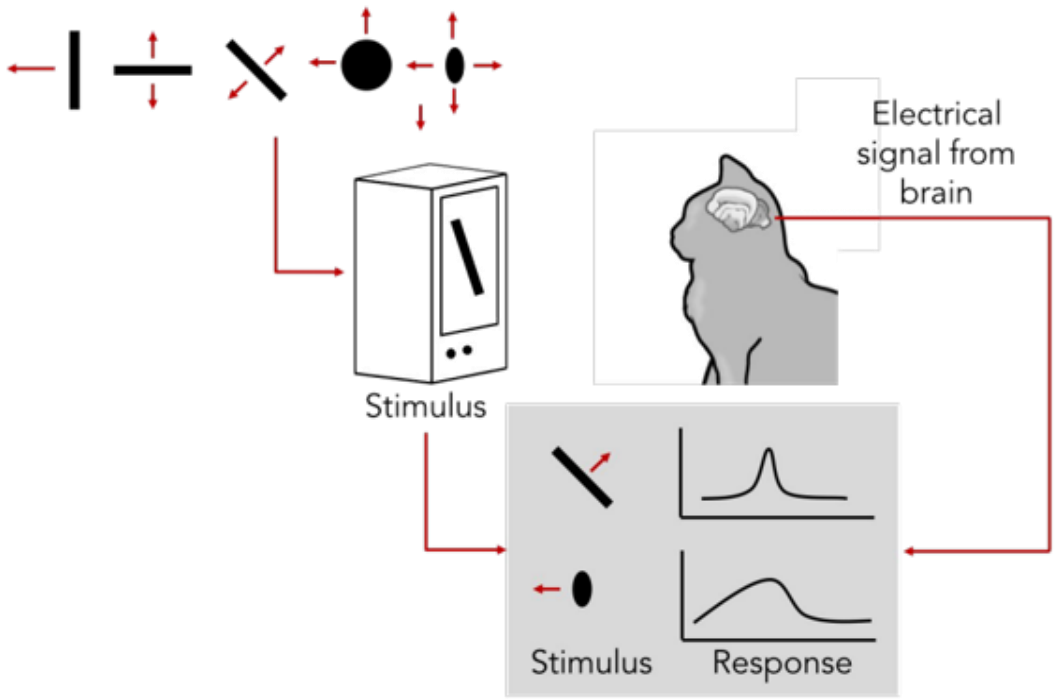
1959

RECEPTIVE FIELDS OF SINGLE NEURONES IN
THE CAT'S STRIATE CORTEX

1962

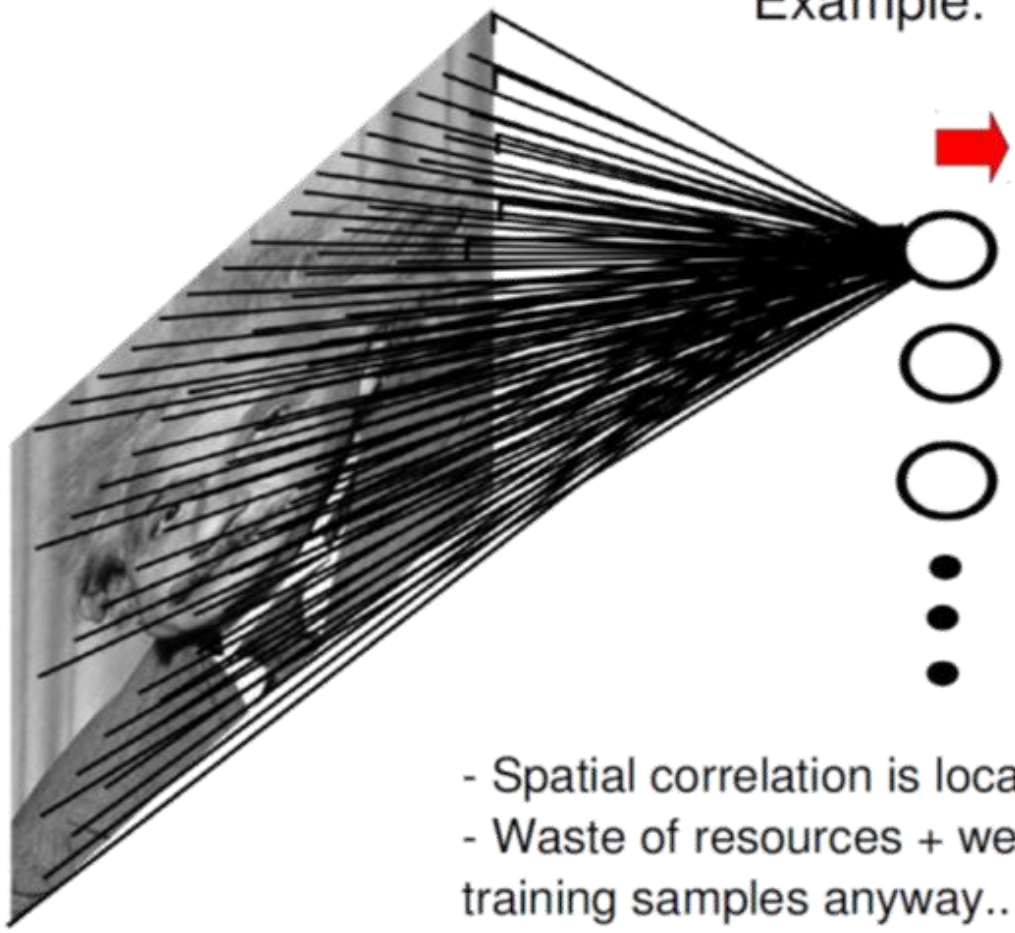
RECEPTIVE FIELDS, BINOCULAR INTERACTION
AND FUNCTIONAL ARCHITECTURE IN
THE CAT'S VISUAL CORTEX

1968...




Cat image by CNX OpenStax is licensed under CC BY 4.0; changes made

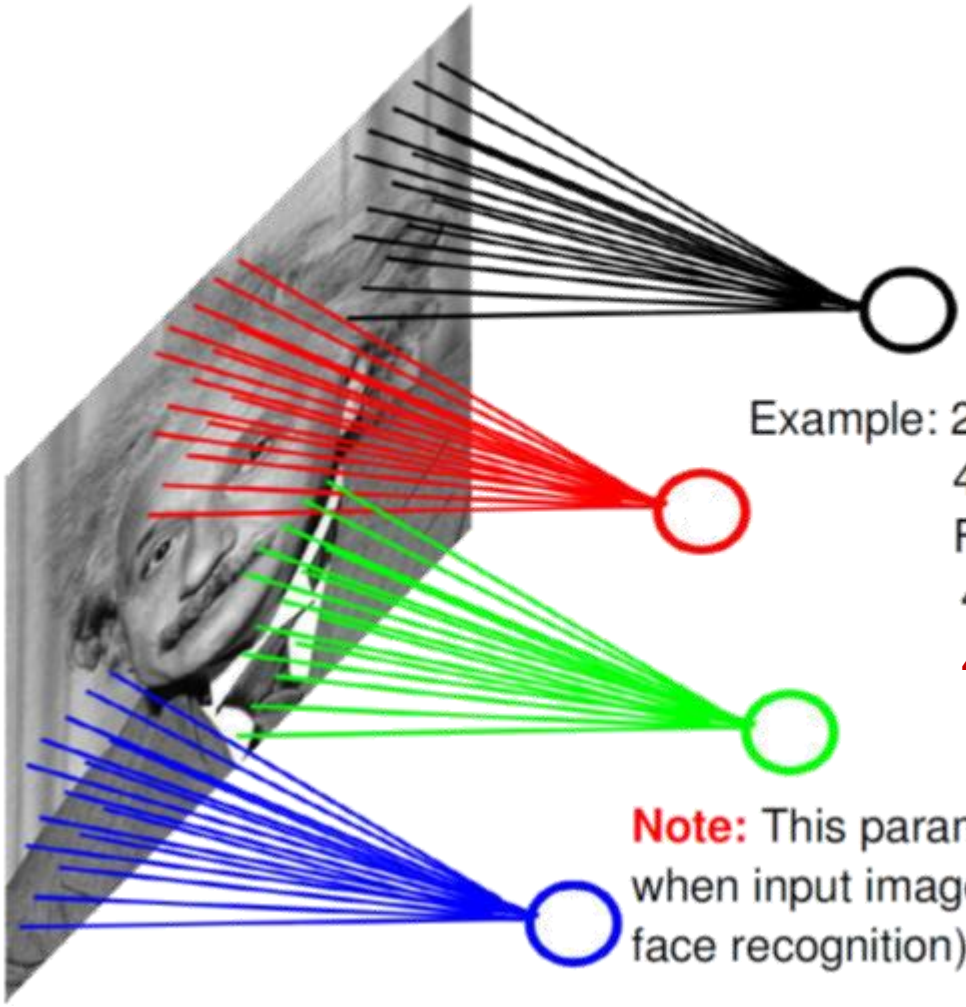
그럼에도 이미지 데이터를 FC로 학습만 잘하면 되는거 아냐?




이웃 픽셀끼리 상관성이 높음에도 불구하고, FC는 모든 픽셀을 전역으로 연결해 이 지역 상관성을 무시!

데이터가 많이 필요하고 연산비용 커짐

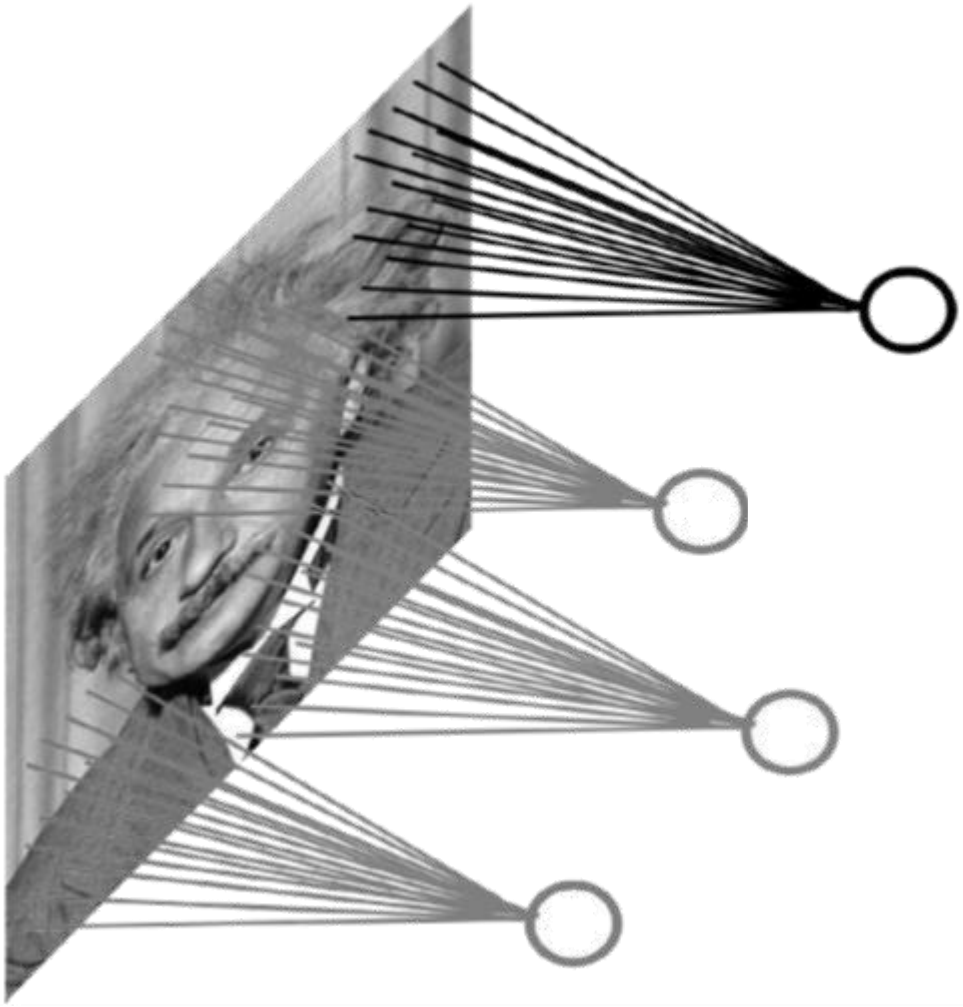
서로 다른 위치의 은닉 유닛이 자기 위치 주변 패치에만 연결



Example: 200x200 image
40K hidden units
Filter size: 10x10
4M parameters

$40{,}000 \times 10 \times 10 = 4{,}000{,}000 \approx 4M$

**Note:** This parameterization is good when input image is registered (e.g., face recognition).



특정위치에 국한되지 않은 필터

즉, 가중치 공유

→하나의 필터가 모든 위치를 옮겨다니면서 처리

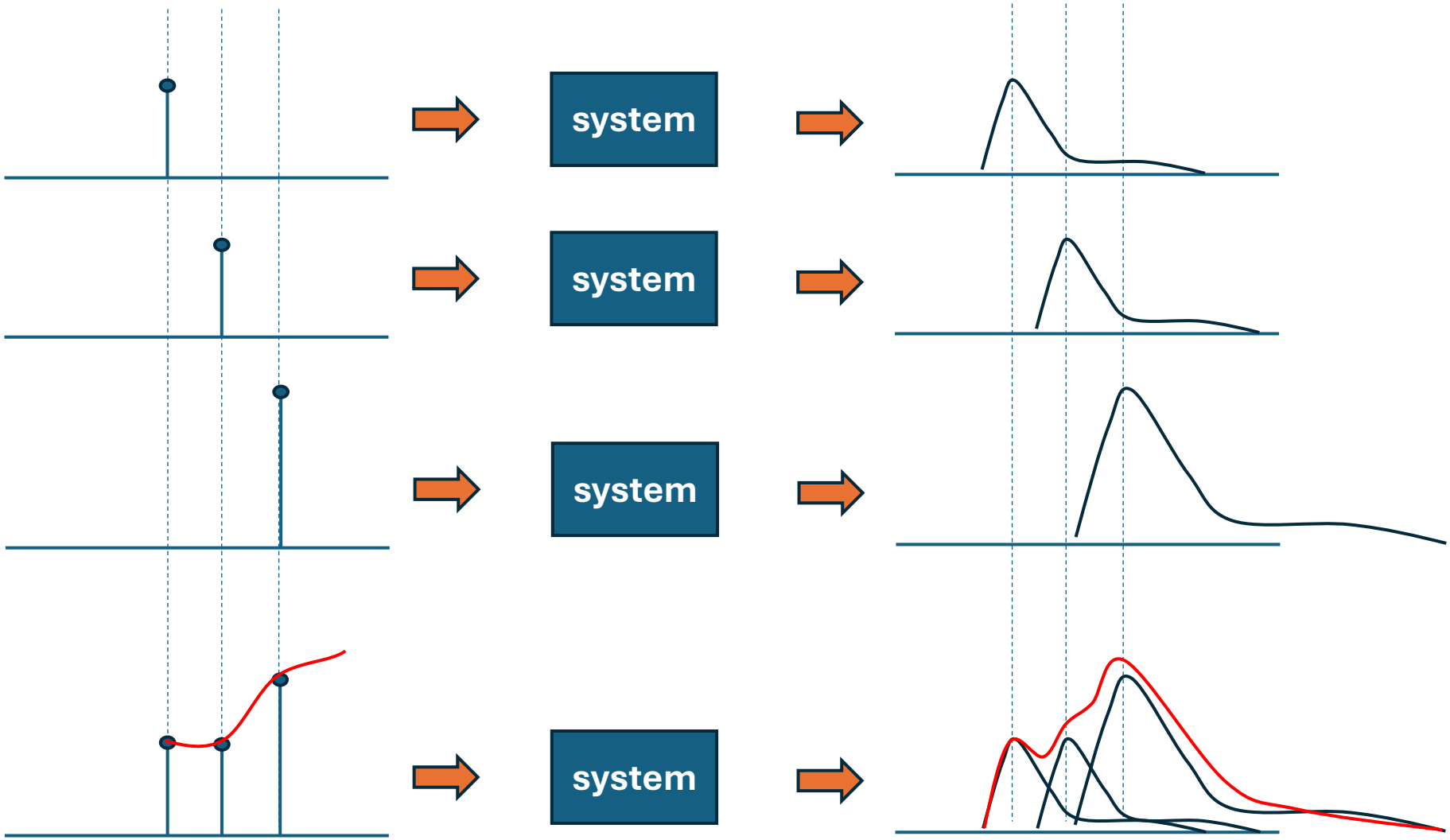
→파라미터 수 10×10=100

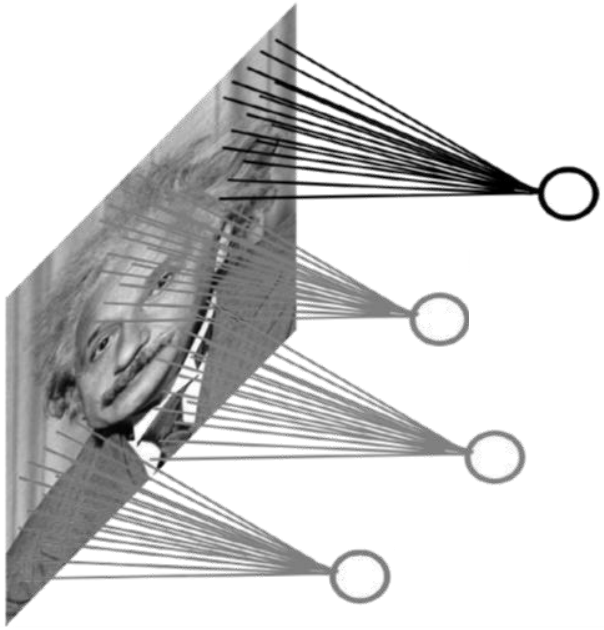
→하나의 필터는 모든 위치에서의 공통된 특징 정보를 찾게됨

→필터수를 늘리면, 찾을 수 있는 특징 정보 종류가 많아지게 됨

# 컨볼루션(신호 중첩합 )원리

AINC
Augmented Intelligence & Cognition Lab.



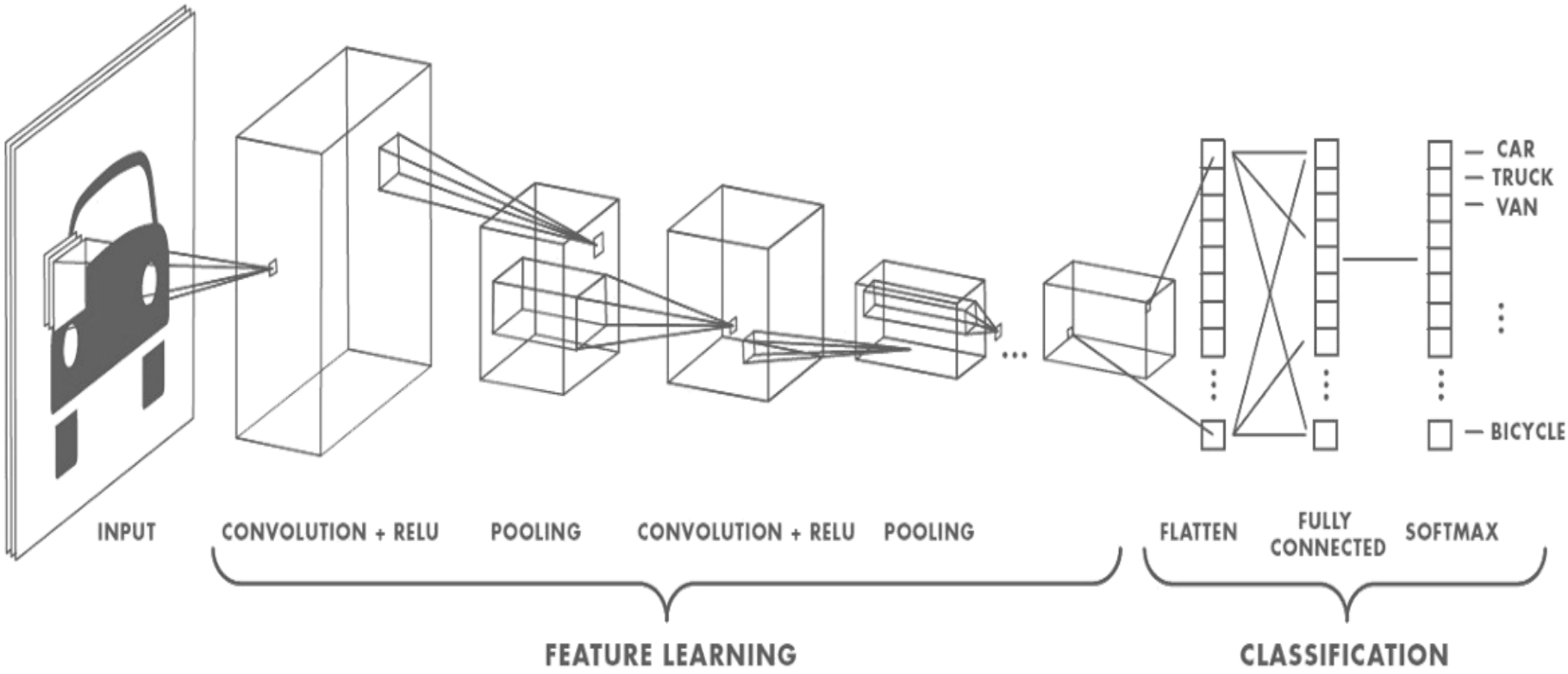


특정위치에 국한되지 않은 필터
즉, 가중치 공유
→하나의 필터가 모든 위치를
옮겨다니면서 처리
→파라미터 수 10×10=100
→하나의 필터는 모든 위치에서의 공통된
특징 정보를 찾게됨
→필터수를 늘리면, 찾을 수 있는 특징 정보
종류가 많아지게 됨

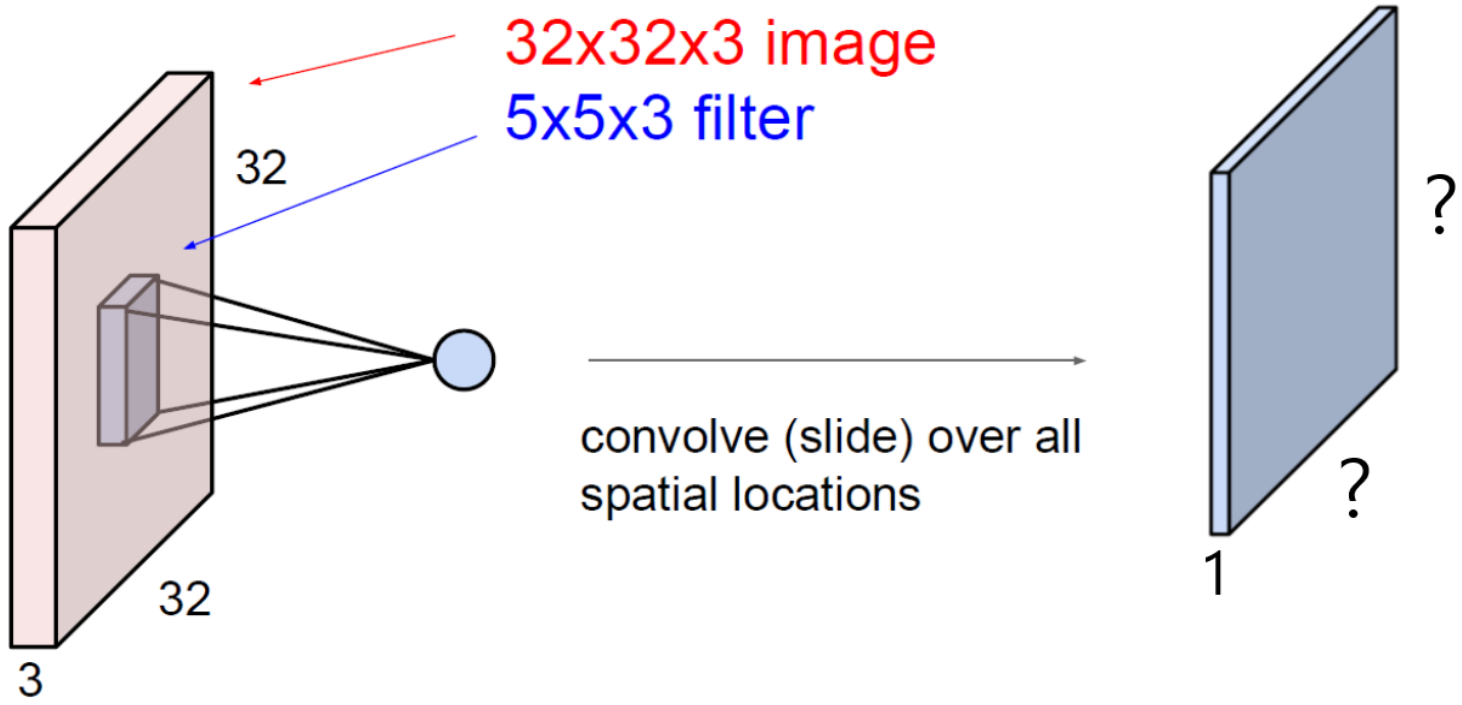
**Convolution :**
**하나의 함수가 다른 함수 위를 이동하면서 가중 평균을 구하는 연산**

**Convolution과 유사한 원리**

# CNN: Convolutional Neural Network


CAR
TRUCK
VAN
BICYCLE
INPUT
CONVOLUTION + RELU
POOLING
CONVOLUTION + RELU
POOLING
FLATTEN
FULLY CONNECTED
SOFTMAX
FEATURE LEARNING
CLASSIFICATION

# Convolution Layer



| | | | | | | |
|---|---|---|---|---|---|---|
| x11 | x12 | x13 | x14 | x15 | x16 | x17 |
| x21 | x22 | x23 | x24 | x25 | x26 | x27 |
| x31 | x32 | x33 | x34 | x35 | x36 | x37 |
| x41 | x42 | x43 | x44 | x45 | x46 | x47 |
| x51 | x52 | x53 | x54 | x55 | x56 | x57 |
| x61 | x62 | x63 | x64 | x65 | x66 | x67 |
| x71 | x72 | x73 | x74 | x75 | x76 | x77 |

7 x 7 input

| | | |
|---|---|---|
| w11 | w12 | w13 |
| w21 | w22 | w23 |
| w31 | w32 | w33 |

3 x 3 filter



y11

| | | | | | | |
|---|---|---|---|---|---|---|
| x11 | x12 | x13 | x14 | x15 | x16 | x17 |
| x21 | x22 | x23 | x24 | x25 | x26 | x27 |
| x31 | x32 | x33 | x34 | x35 | x36 | x37 |
| x41 | x42 | x43 | x44 | x45 | x46 | x47 |
| x51 | x52 | x53 | x54 | x55 | x56 | x57 |
| x61 | x62 | x63 | x64 | x65 | x66 | x67 |
| x71 | x72 | x73 | x74 | x75 | x76 | x77 |

7 x 7 input

| | | |
|---|---|---|
| w11 | w12 | w13 |
| w21 | w22 | w23 |
| w31 | w32 | w33 |

3 x 3 filter



| | |
|---|---|
| y11 | y12 |

| | | | | | | |
|---|---|---|---|---|---|---|
| x11 | x12 | x13 | x14 | x15 | x16 | x17 |
| x21 | x22 | x23 | x24 | x25 | x26 | x27 |
| x31 | x32 | x33 | x34 | x35 | x36 | x37 |
| x41 | x42 | x43 | x44 | x45 | x46 | x47 |
| x51 | x52 | x53 | x54 | x55 | x56 | x57 |
| x61 | x62 | x63 | x64 | x65 | x66 | x67 |
| x71 | x72 | x73 | x74 | x75 | x76 | x77 |

7 x 7 input

| | | |
|---|---|---|
| w11 | w12 | w13 |
| w21 | w22 | w23 |
| w31 | w32 | w33 |

3 x 3 filter



| | | | | |
|---|---|---|---|---|
| y11 | y12 | y13 | y14 | y15 |

| | | | | | | |
|---|---|---|---|---|---|---|
| x11 | x12 | x13 | x14 | x15 | x16 | x17 |
| x21 | x22 | x23 | x24 | x25 | x26 | x27 |
| x31 | x32 | x33 | x34 | x35 | x36 | x37 |
| x41 | x42 | x43 | x44 | x45 | x46 | x47 |
| x51 | x52 | x53 | x54 | x55 | x56 | x57 |
| x61 | x62 | x63 | x64 | x65 | x66 | x67 |
| x71 | x72 | x73 | x74 | x75 | x76 | x77 |

7 x 7 input

| | | |
|---|---|---|
| w11 | w12 | w13 |
| w21 | w22 | w23 |
| w31 | w32 | w33 |

3 x 3 filter



| | | | | |
|---|---|---|---|---|
| y11 | y12 | y13 | y14 | y15 |
| y21 | | | | |

| | | | | | | |
|---|---|---|---|---|---|---|
| x11 | x12 | x13 | x14 | x15 | x16 | x17 |
| x21 | x22 | x23 | x24 | x25 | x26 | x27 |
| x31 | x32 | x33 | x34 | x35 | x36 | x37 |
| x41 | x42 | x43 | x44 | x45 | x46 | x47 |
| x51 | x52 | x53 | x54 | x55 | x56 | x57 |
| x61 | x62 | x63 | x64 | x65 | x66 | x67 |
| x71 | x72 | x73 | x74 | x75 | x76 | x77 |

7 x 7 input

| | | |
|---|---|---|
| w11 | w12 | w13 |
| w21 | w22 | w23 |
| w31 | w32 | w33 |

3 x 3 filter



| | | | | |
|---|---|---|---|---|
| y11 | y12 | y13 | y14 | y15 |
| y21 | y22 | y23 | y24 | y25 |
| y31 | y32 | y33 | y34 | y35 |
| y41 | y42 | y43 | y44 | y45 |
| y51 | y52 | y53 | y54 | y55 |

**with stride 1**

| | | | | | | |
|---|---|---|---|---|---|---|
| x11 | x12 | x13 | x14 | x15 | x16 | x17 |
| x21 | x22 | x23 | x24 | x25 | x26 | x27 |
| x31 | x32 | x33 | x34 | x35 | x36 | x37 |
| x41 | x42 | x43 | x44 | x45 | x46 | x47 |
| x51 | x52 | x53 | x54 | x55 | x56 | x57 |
| x61 | x62 | x63 | x64 | x65 | x66 | x67 |
| x71 | x72 | x73 | x74 | x75 | x76 | x77 |

7 x 7 input

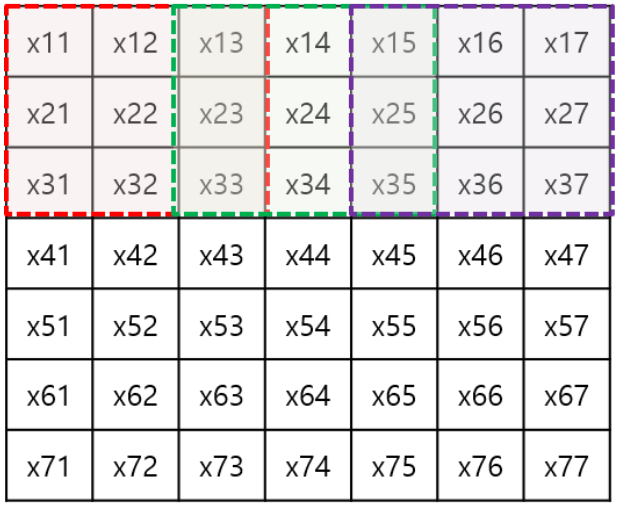
| | | |
|---|---|---|
| w11 | w12 | w13 |
| w21 | w22 | w23 |
| w31 | w32 | w33 |

3 x 3 filter





**with stride 2**




(N x N) input




(F x F) filter

Output size:

**(N - F) / stride + 1**

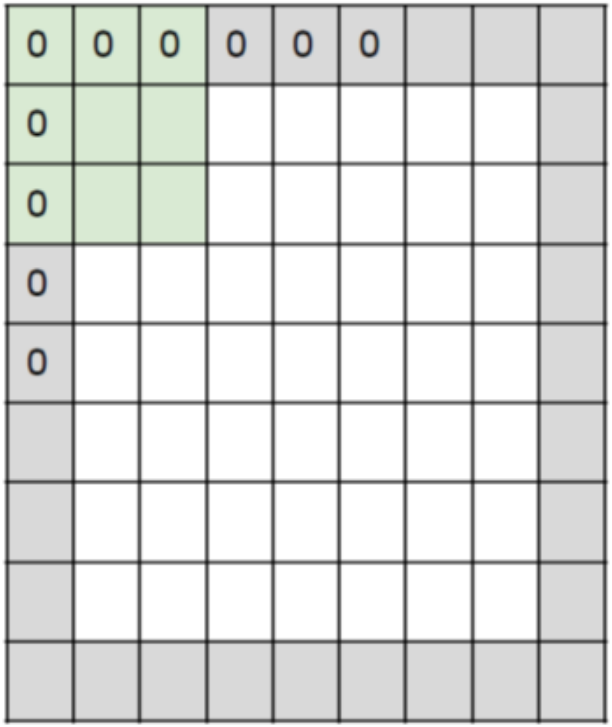
e.g. N = 7, F = 3:

stride 1 => (7 - 3)/1 + 1 = 5

stride 2 => (7 - 3)/2 + 1 = 3

stride 3 => (7 - 3)/3 + 1 = 2.33

# In practice: Common to zero pad the border



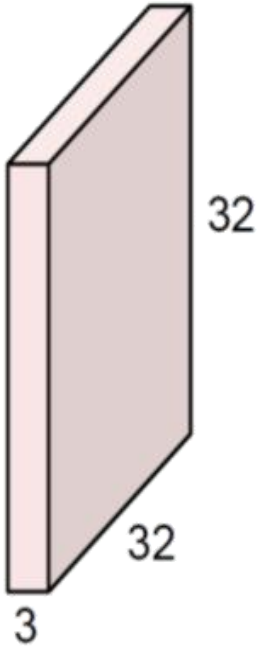

e.g. input 7x7
**3x3** filter, applied with **stride 1**
**pad with 1 pixel** border => what is the output?

**7x7 output!**
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

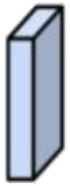
# Convolution Layer

Filters always extend the full depth of the input volume

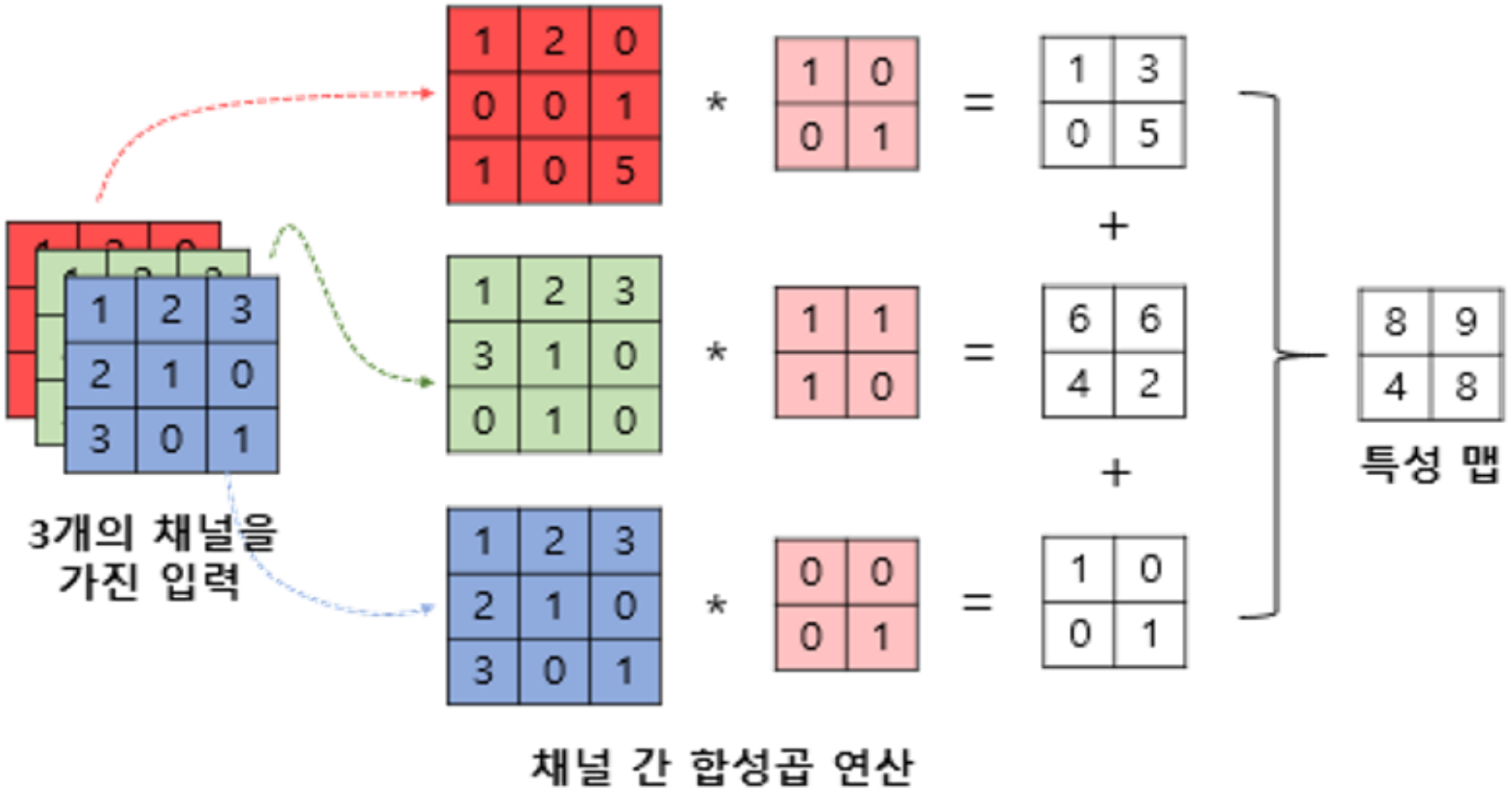
32x32x3 image




5x5x3 filter



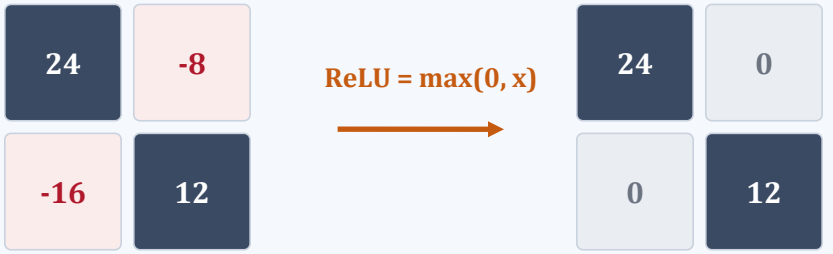
**Convolve** the filter with the image
i.e. “slide over the image spatially, computing dot products”


1 2 3
2 1 0
3 0 1
3개의 채널을
가진 입력
1 2 0
0 0 1
1 0 5
1 2 3
3 1 0
0 1 0
1 2 3
2 1 0
3 0 1
*
1 0
0 1
=
1 3
0 5
+
1 1
1 0
=
6 6
4 2
+
0 0
0 1
=
1 0
0 1
8 9
4 8
특성 맵
채널 간 합성곱 연산

# 합성곱 뒤엔 ReLU — 활성화맵

ReLU의 경우 0으로 잘라 '패턴 있음 ' 만 남김

## 수치로 보기 — 특징맵 → ReLU → 활성화맵

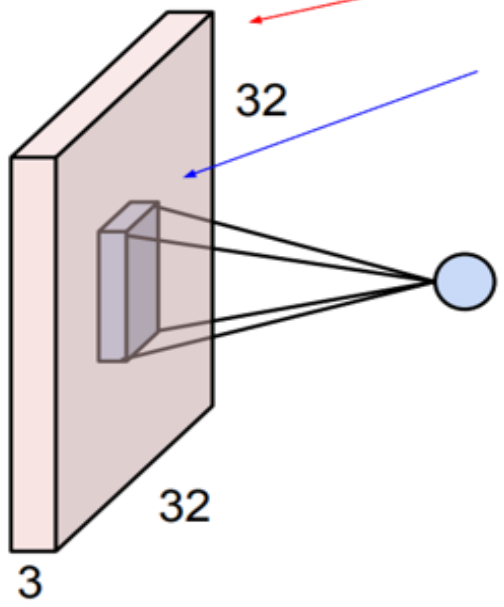



특징맵 (음수 = 반대 방향 반응)

활성화맵 (있음만 남김)

- 관례 블록: Conv → (BN) → ReLU
- 3주차 BN이 여기 끼어듭니다

# Convolution Layer




32x32x3 image

5x5x3 filter $w$

**1 number:**
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

# 합성곱도 결국 Z = WX + B

합성곱은 '아주 특별한 모양의 행렬곱'입니다 — 4주차의 골격이 여기서도 그대로 성립합니다.

## 펼쳐 보면 — 희소 + 공유 행렬

| | | | | | | |
|---|---|---|---|---|---|---|
| $w_1$ | $w_2$ | $w_3$ | | | | $w_1x_1 + w_2x_2 + w_3x_3$ |
| | $w_1$ | $w_2$ | $w_3$ | | | $w_1x_2 + w_2x_3 + w_3x_4$ |
| | | $w_1$ | $w_2$ | $w_3$ | | $w_1x_3 + w_2x_4 + w_3x_5$ |
| | | | $w_1$ | $w_2$ | $w_3$ | $w_1x_4 + w_2x_5 + w_3x_6$ |

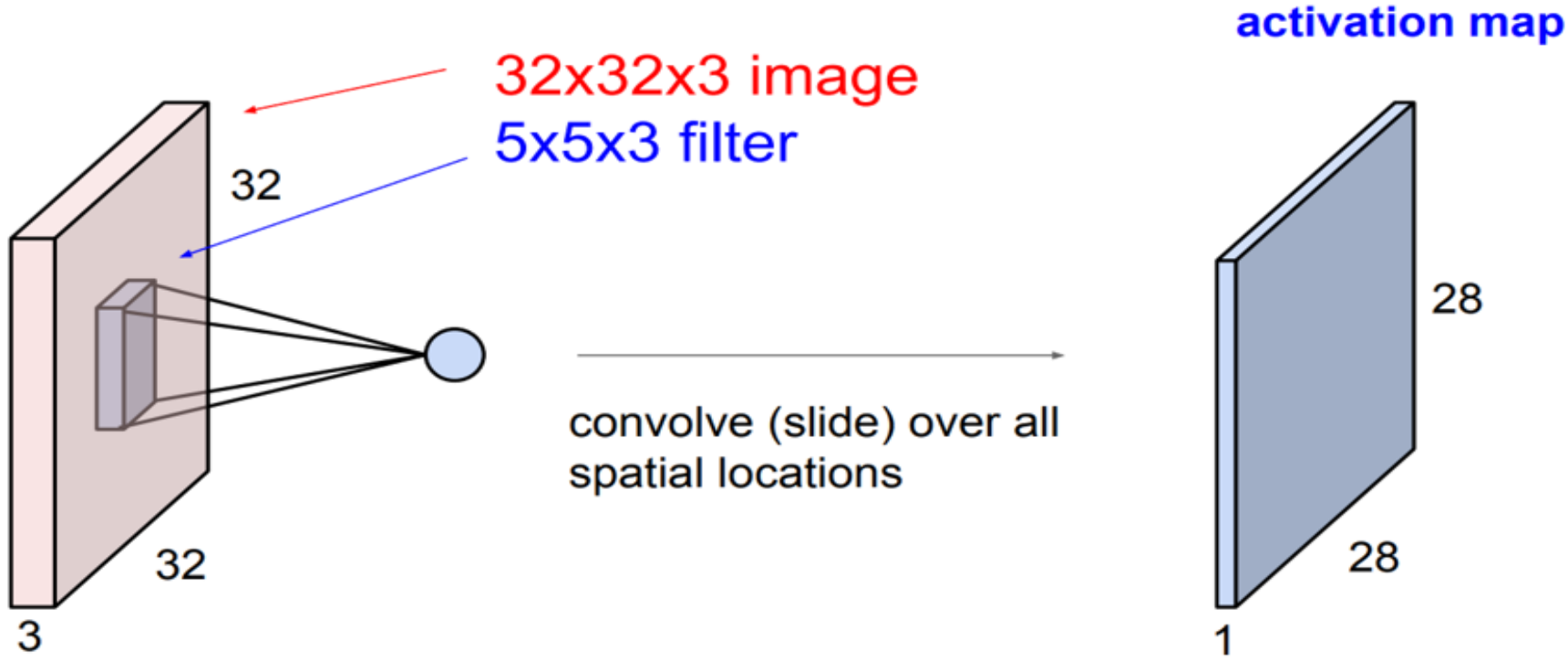
행마다 같은 $w_1w_2w_3$가 자리만 밀려 반복 — 나머지는 전부 0

## 두 제약의 정체

- **0이 많다 = 국소 연결 (희소성)**
- **행마다 같은 값 = 가중치 공유**
- 그래서 파라미터 폭발이 사라짐 — FC의 특수형이지 다른 세계가 아님
- 역전파도 4주차 $(AB)^T=B^TA^T$ 규칙 그대로 작동

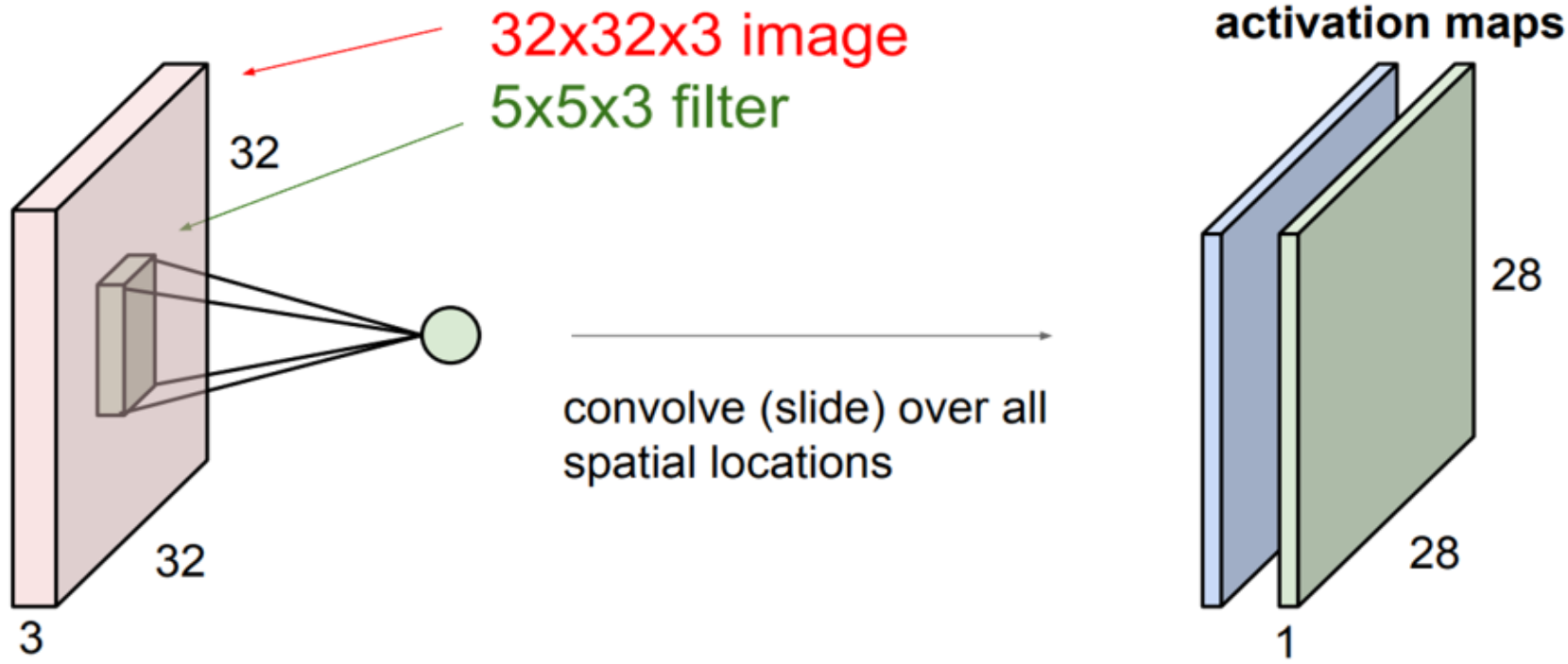
**표현학습 축** — 퍼셉트론·FC·CNN 모두 Z=WX+B 하나의 변주입니다. 제약을 어떻게 두는가만 다릅니다.

# Convolution Layer

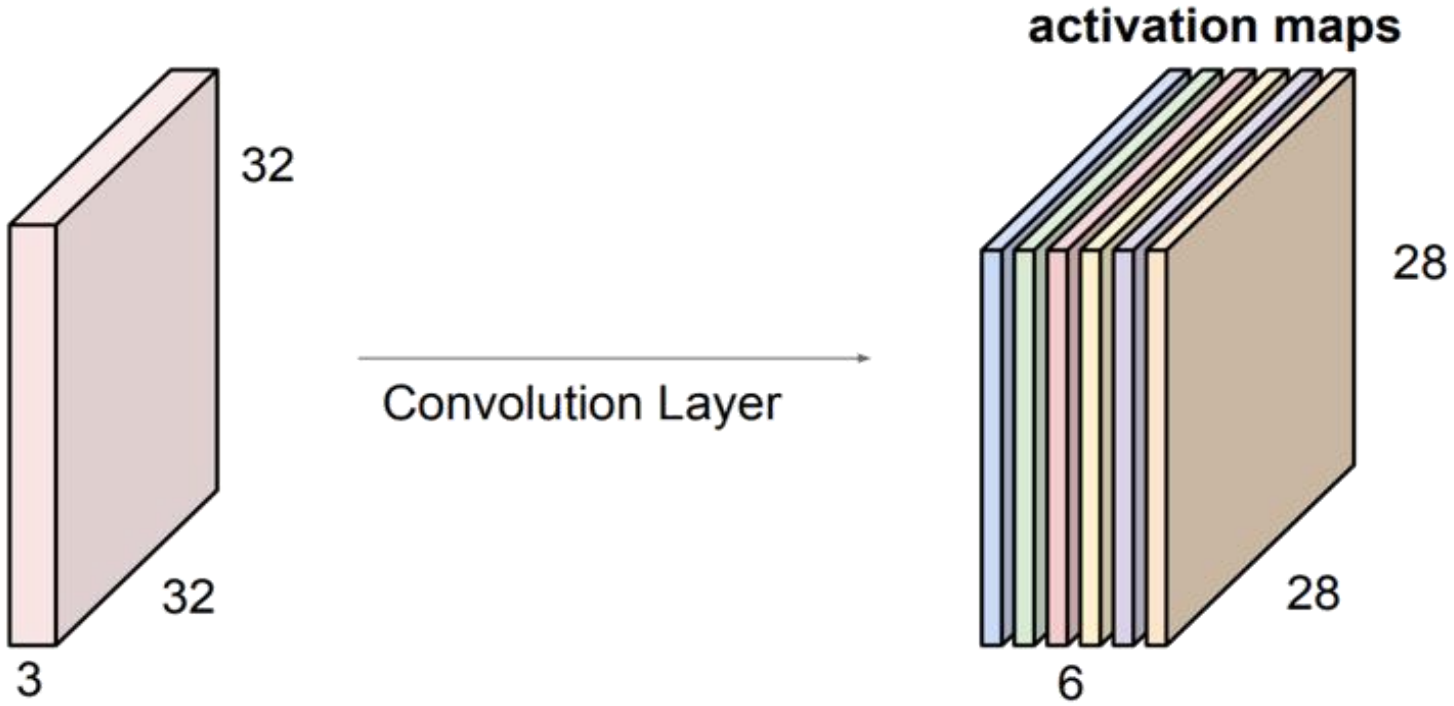


# Convolution Layer

consider a second, green filter



For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

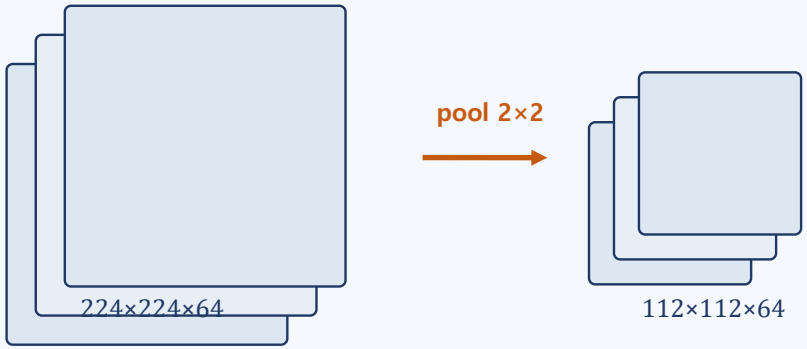



We stack these up to get a "new image" of size 28x28x6!

# 풀링 — 표현을 작게

풀링은 특징맵을 줄여 계산을 가볍게 하고, 작은 위치 변화에 강인하게 만듦

## 각 채널을 독립적으로 축소




공간 크기만 절반 · 채널 수는 그대로

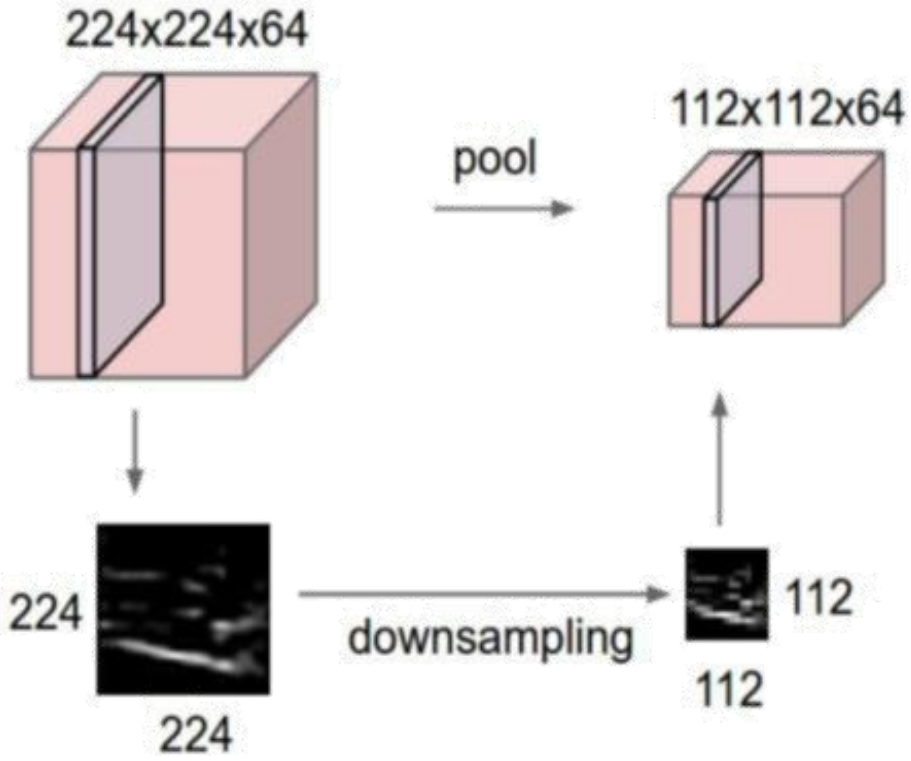
## 왜 쓰나

- **계산·메모리 ↓** 크기가 절반으로
- **위치 변화에 강인** 약간 움직여도 같은 결과
- **채널 수 유지** 공간 크기만 축소

**풀링엔 학습 파라미터가 없음**

# Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:



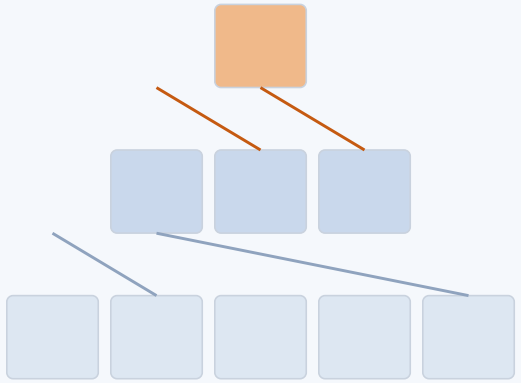




# 수용영역 — 층을 쌓으면 넓게 본다

3×3만 쌓아도 깊어질수록 '넓은 문맥'을 보게됨

### 3×3 두 번 = 5×5 효과 (1D 단면)



conv3 또 — 1칸이 5칸 봄

conv3 후 — 각 칸이 3칸 봄

입력 — 5칸

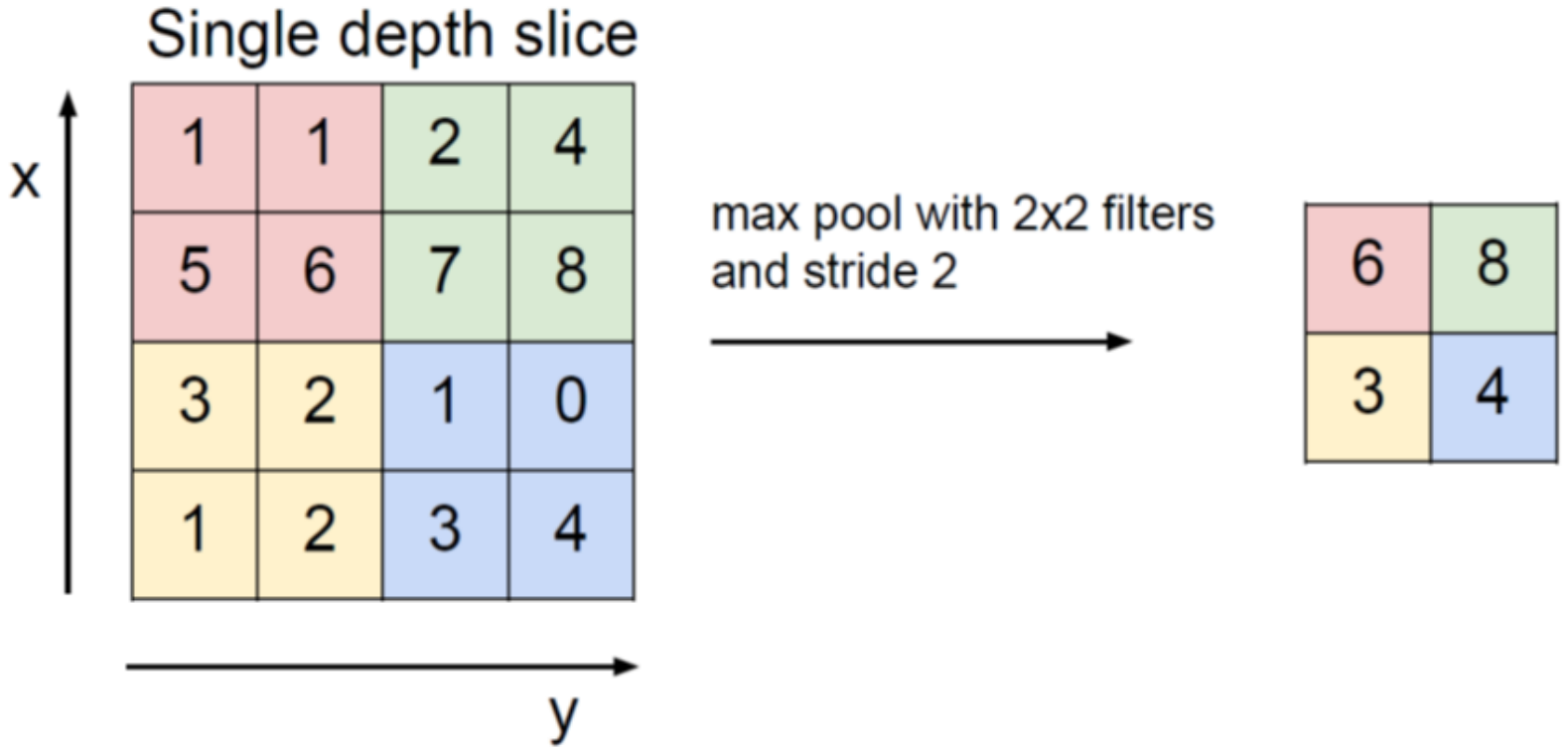
### 수용영역(RF) 추적표

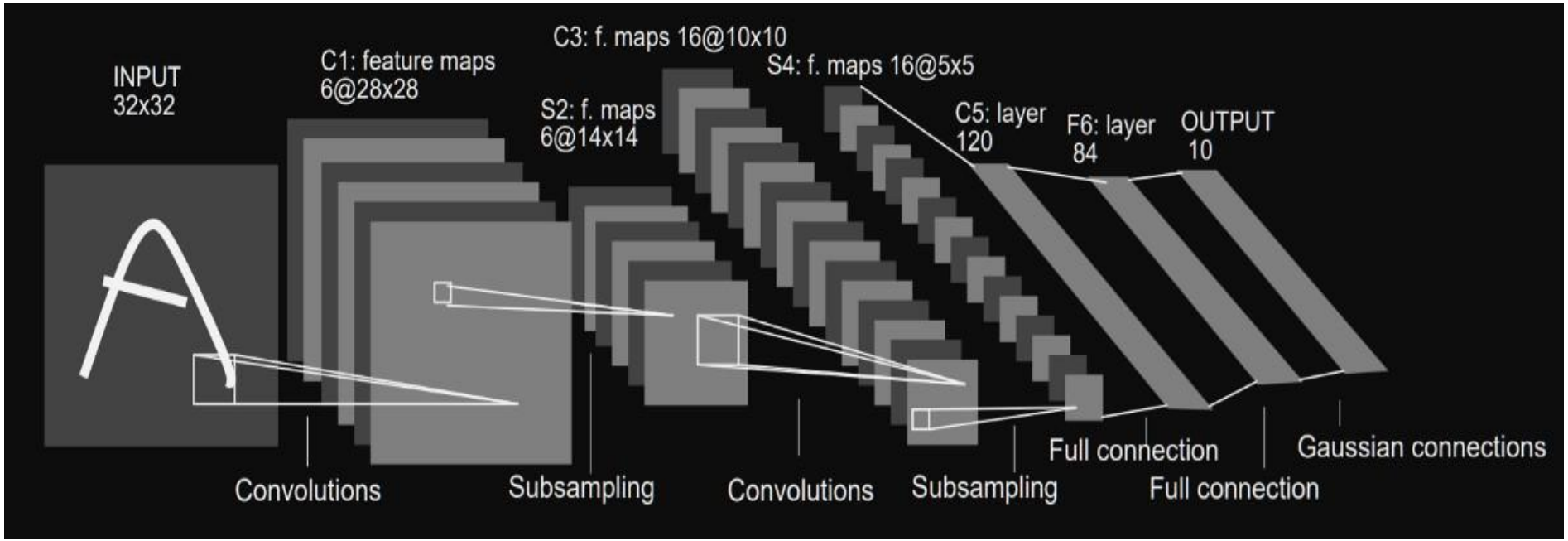
| 층 | 연산 | RF |
|---|---|---|
| 1 | conv 3×3 | 3 |
| 2 | conv 3×3 | 5 |
| 3 | pool 2×2 | 6 |
| 4 | conv 3×3 | **10** |

풀링이 '보폭'을 2배로 → 이후 conv의 RF 증가도 2배

**작은 필터를 깊게** 가 큰 필터 한 번보다 파라미터는 적고(2×9<25) 표현력은 높은 이유

# MAX POOLING


Single depth slice
x
y
1 1 2 4
5 6 7 8
3 2 1 0
1 2 3 4
max pool with 2x2 filters
and stride 2
6 8
3 4


INPUT
32x32
C1: feature maps
6@28x28
S2: f. maps
6@14x14
C3: f. maps 16@10x10
S4: f. maps 16@5x5
C5: layer
120
F6: layer
84
OUTPUT
10
Convolutions
Subsampling
Convolutions
Subsampling
Full connection
Full connection
Gaussian connections

# MNIST 데이터셋을 활용한 합성곱 신경망 실습

# Thank you!

본 교재는 2025&2026학년도 과학기술정보통신부 및 정보통신기획평가원의 'SW중심대학사업' 지원을 받아 제작되었습니다